

Project Report
On
“Food Delivery Web Application with CI/CD and Cloud
Deployment”

Master of Computer Applications

Submitted By

Daneshwari Hanji	2GI24MC024
Disha Birje	2GI24MC025
Prachi Patil	2GI24MC055
Sakshi Sankpal	2GI24MC085

Under the Guidance of

Prof. Pavan Mitragotri



Department of Master of Computer Applications

Karnatak Law Society's

GOGTE INSTITUTE OF TECHNOLOGY

Udyambag, Belagavi -590008

Karnatak Law Society's

GOGTE INSTITUTE OF TECHNOLOGY

Udyambag, Belagavi -590008

2025-2026



Department of Master of Computer Applications

Certificate

This is to certify that students of 3rd semester with above mentioned names and USN have successfully completed their course project work entitled “Weather App Broadcasting” and have orally presented the same. This case study report is submitted to partial fulfil the requirement of Outcome Based Education (OBE) practiced in the institute.

USN	Name	Marks
2GI24MC021	Chaitra B. Biradar	
2GI24MC022	Chaitra S. Sidamani	
2GI24MC039	Laxmi S. Mullur	
2GI24MC089	Shivani S. Kotabagi	

Name and signature of examiner with date

Marks obtained

Index

Sl.No	Contents	Pages
1	Introduction	01
2	OBJECTIVES OF THE PROJECT	02
3	DevOps Workflow / Pipeline	03
4	TOOLS AND TECHNOLOGIES USED	04-07
5	Screenshots	08-10
6	Challenges Faced	11
7	Conclusion	12
8	References	13

1.Introduction

With the rapid growth of internet usage and mobile technology, online food delivery systems have become an essential part of daily life. Traditional food ordering methods such as phone calls or in-person visits are inefficient, timeconsuming, and prone to errors. Online food delivery platforms solve these problems by providing a centralized digital interface for customers and restaurants.

with application development, modern software systems require **automation, scalability, and reliability**. DevOps is a methodology that bridges the gap between development and operations teams by introducing automation tools and best practices. CI/CD pipelines allow applications to be built, tested, and deployed automatically whenever code changes are made.

This project focuses on building a Food Delivery Web Application while implementing a **DevOps-driven deployment model** using Jenkins, Docker, and AWS. The goal is to demonstrate how DevOps tools can significantly improve software delivery speed, consistency, and reliability.

2. OBJECTIVES OF THE PROJECT

The objectives of the Food Delivery Web Application with DevOps Implementation are as follows:

1. To design and develop a full-stack web application that allows users to browse restaurants, view menus, add food items to a cart, and place orders online.
2. To implement DevOps principles in the software development lifecycle to achieve faster, reliable, and automated application deployment.
3. To build an automated CI/CD pipeline using Jenkins that continuously integrates source code changes and deploys the application without manual intervention.
4. To use Docker for containerization of both frontend and backend applications to ensure consistency across development and production environments.
5. To deploy the application on cloud infrastructure (AWS EC2) to achieve high availability, scalability, and real-world hosting experience.
6. To integrate MongoDB Atlas as a cloud-based database solution for secure, scalable, and efficient data storage.
7. To utilize Cloudinary for image management, enabling efficient storage, retrieval, and delivery of restaurant and menu images.
8. To ensure automated build and deployment processes, reducing human errors and deployment time.
9. To gain hands-on experience with industry-standard DevOps tools such as GitHub, Jenkins, Docker, and AWS.
10. To enhance system reliability and maintainability through containerized deployment and automated pipelines.
11. To understand real-time deployment workflows followed in modern software development organizations.

3. DevOps Workflow / CI-CD Pipeline

The project follows an automated DevOps workflow to ensure continuous integration and continuous deployment of the application. Whenever a developer makes changes to the application source code and pushes them to the central repository, the pipeline is automatically triggered. This automation removes the need for manual intervention and ensures that every change is processed in a consistent manner.

Once the pipeline is triggered, the application build process starts. During this phase, both the backend and frontend components are prepared for deployment. All required dependencies are installed, and the application is verified to ensure that it is ready to run. This step helps in identifying errors early and maintains code quality throughout the development lifecycle.

After a successful build, the application is packaged into container images. Containerization ensures that the application runs in the same environment regardless of where it is deployed. This eliminates compatibility issues and provides a reliable way to deploy the application across different systems.

The generated container images are then stored in a centralized image repository. This repository acts as a versioned storage location for application images, allowing the latest and stable versions to be easily retrieved during deployment. Storing images centrally also enables seamless updates and rollback if required.

In the deployment phase, the cloud server automatically pulls the latest available images from the repository. Any previously running instances of the application are stopped, and new instances are started using the updated images. This ensures that the live application always reflects the most recent changes made by the development team.

Finally, the application is hosted on the cloud and made accessible to users through a public URL. The backend services handle application logic and data processing, while the frontend provides a user-friendly interface. With this fully automated pipeline, every code update results in an automatic build, deployment, and update on the cloud server, ensuring fast delivery, reduced errors, and efficient application management.

4. TOOLS AND TECHNOLOGIES USED

4.1 Git (Version Control System)

Git is a distributed version control system used to track changes in the source code. It allows developers to collaborate efficiently, maintain different versions of the application, and rollback changes if required.

Purpose in Project

- Track source code changes
- Maintain version history
- Enable collaboration
- Integrate with Jenkins for CI/CD

Git Commands Used

`git init`

`git status`

`git add .`

`git commit -m "Initial commit"`

`git branch`

`git pull origin main`

Benefits

- Prevents code conflicts
- Maintains history of changes
- Enables rollback to previous versions

4.2 GitHub (Source Code Repository)

GitHub is a cloud-based Git repository hosting service. It stores the project code and acts as the trigger point for the CI/CD pipeline.

Purpose in Project

- Store backend and frontend source code
- Trigger Jenkins pipeline on code push
- Centralized code management

Features Used

- Remote repository
- Branch management
- Webhooks / Poll SCM integration with Jenkins

4.3 Jenkins (Continuous Integration and Continuous Deployment Tool)

Jenkins is an open-source automation server used to implement CI/CD pipelines. It automates the entire build and deployment process whenever changes are pushed to GitHub.

Purpose in Project

- Automate build process
- Automate Docker image creation
- Deploy containers automatically
- Reduce manual intervention

Jenkins Pipeline Stages

- Clone repository
- Build backend Docker image
- Deploy backend container
- Build frontend Docker image
- Deploy frontend container

Jenkins Commands Used

```
git clone https://github.com/username/repository.git
```

```
docker build -t food-backend-image .
```

```
docker stop food-backend-container
```

```
docker rm food-backendcontainer
```

```
docker run -d -p 5000:5000 food-
```

Benefits

- Faster deployments
- Reduced human errors
- Consistent build process
- Automatic rollback on failure

4.4 Docker Hub(Containerization Platform)

DockerHub is used to containerize both frontend and backend applications, ensuring consistency across development, testing, and production environments.

Purpose in Project

- Package application with dependencies
- Run application in isolated containers
- Ensure environment consistency

Docker Commands Used

```
docker --version docker build -t food-backend-image
```

```
. docker build -t food-frontend-image .
```

```
docker images docker ps docker ps -a docker
```

```
stop container_name docker rm container_name
```

```
docker run -d -p 5000:5000 food-backendimage
```

```
docker run -d -p 80:80 food-frontendimage
```

Benefits

- Lightweight deployment
- Faster startup time
- Easy scalability
- Platform independence

4.5 AWS EC2 (Cloud Infrastructure)

Amazon Elastic Compute Cloud (EC2) is used to host the Docker containers in a cloud environment.

Purpose in Project

- Host backend and frontend containers
- Provide public access using IP address
- Enable scalable cloud deployment

AWS Commands Used (Linux EC2 Instance)

```
sudo apt update  
sudo apt install docker.io -y
```

```
sudo systemctl start docker
```

```
sudo systemctl enable docker  
docker ps
```

Benefits

- High availability
- On-demand scalability
- Secure hosting
- Industry-standard cloud platform

4.6 MongoDB Atlas (Cloud Database)

MongoDB Atlas is a fully managed NoSQL database service used to store application data.

Purpose in Project

- Store users, restaurants, menus, and orders
- Cloud-based data storage
- Automatic backup and scaling

MongoDB Features Used

Cloud connection string

- Environment variables for security

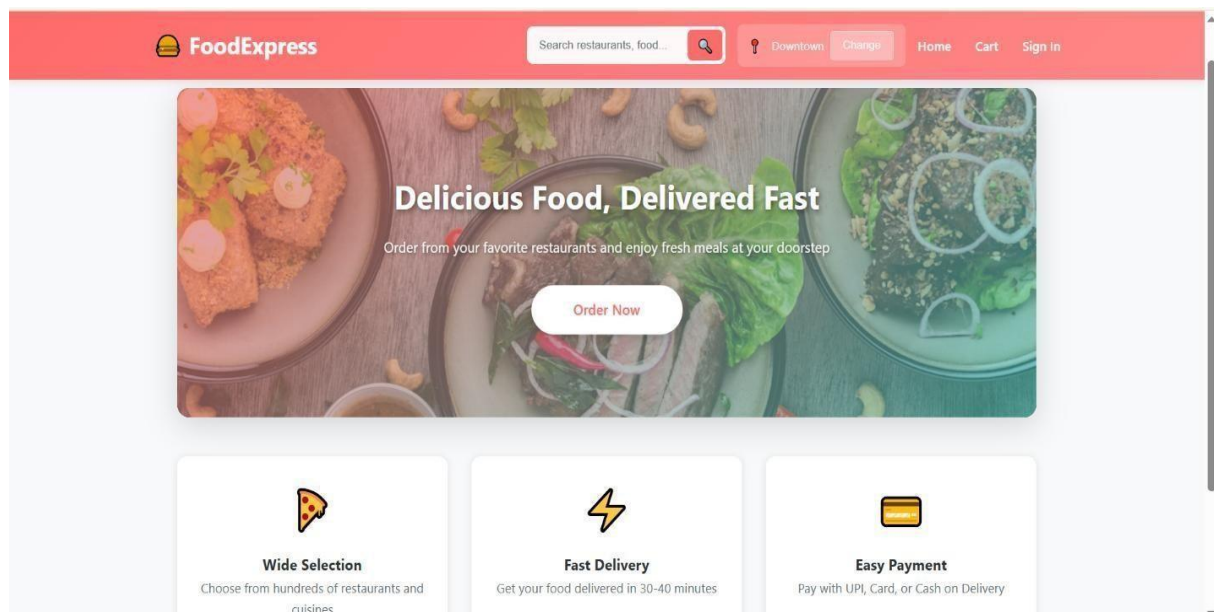
5. Screenshots:

The first screenshot shows the 'Create Account' page. It has a title 'Create Account' with a clapping hands emoji. Below it is the text 'Sign up to get started'. There are four input fields: 'Full Name', 'Email Address', 'Password', and 'Confirm Password'. A red 'Sign Up' button is at the bottom. A link 'Already have an account? Sign In' is at the very bottom.

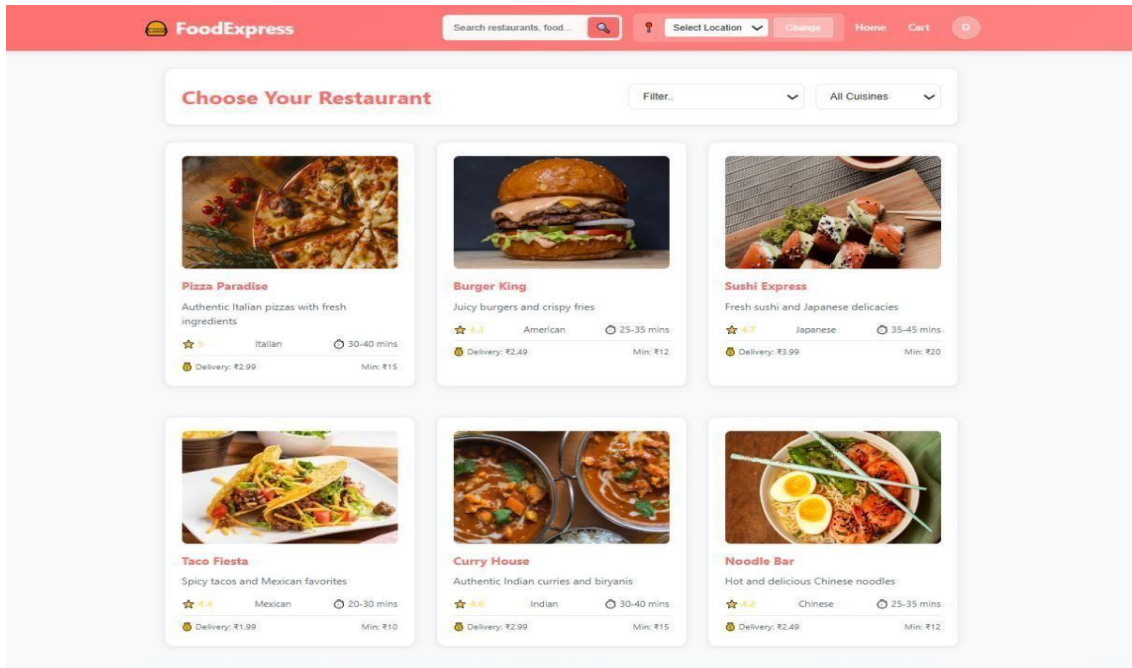
The second screenshot shows the 'Welcome Back' login page. It has a title 'Welcome Back' with a clapping hands emoji. Below it is the text 'Sign in to continue'. There are two input fields: 'Email Address' and 'Password'. A red 'Sign In' button is in the center. A link 'Don't have an account? Sign Up' is at the bottom.

Screenshot 1: User Registration page allowing new users to create an account using valid credentials.

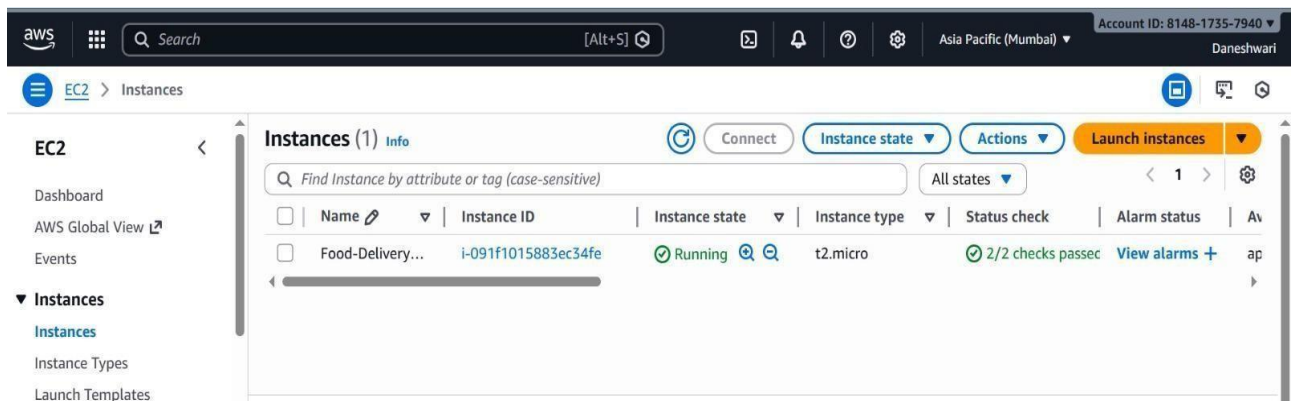
Screenshot 2: Login page enabling registered users to securely access the application.



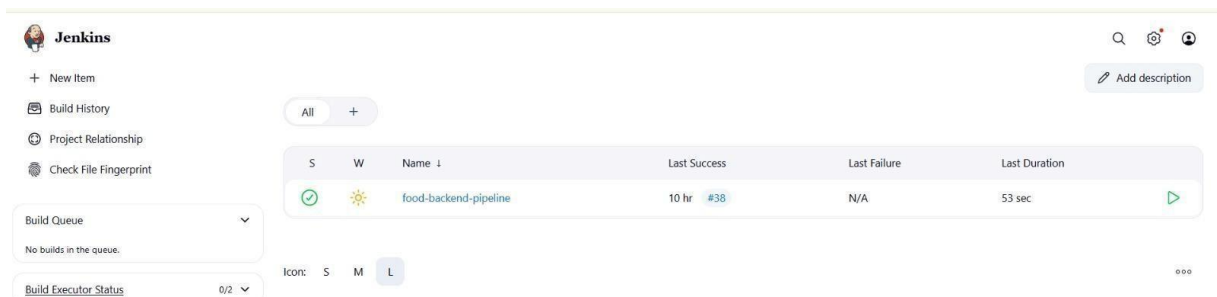
Screenshot 3: Home page displaying the application overview, services, and navigation options.



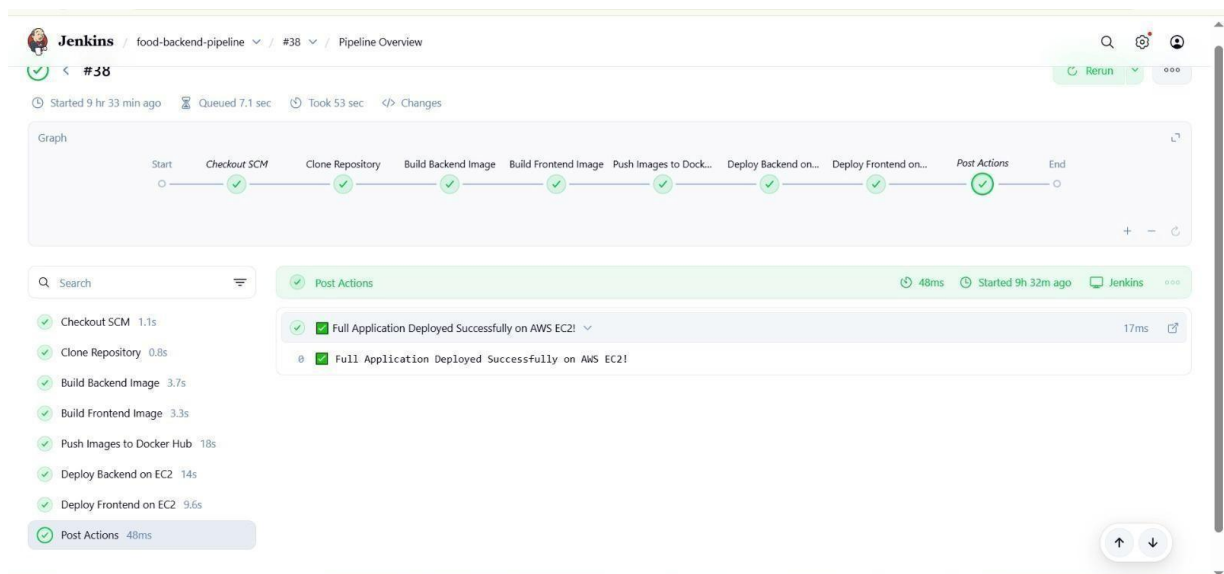
Screenshot4: Restaurant page showing available restaurants along with menus and food items.



Screenshot 5. AWS EC2 instance created and running for hosting the application.



Screenshot 6. Jenkins CI/CD pipeline showing automated build and deployment process.



Screenshot7: This screenshot shows the successful execution of the Jenkins CI/CD pipeline, including code checkout, Docker image build, image push to Docker Hub, and automated deployment on AWS EC2.

The screenshot shows the Docker Hub interface for a user named 'birjedisha'. The 'Repositories' section is active, displaying a list of repositories within the 'birjedisha' namespace. The table lists two repositories: 'birjedisha/food-frontend' and 'birjedisha/food-backend'. Both repositories are public and have been pushed about 10 hours ago. The 'Scout' column for both repositories is 'Inactive'.

Name	Last Pushed	Contains	Visibility	Scout
birjedisha/food-frontend	about 10 hours ago	IMAGE	Public	Inactive
birjedisha/food-backend	about 10 hours ago	IMAGE	Public	Inactive

Screenshot8: This screenshot displays the Docker Hub repositories containing the frontend and backend Docker images pushed automatically through the Jenkins CI/CD pipeline.

6. Challenges Faced

1. Jenkins Credential Configuration Issues

Configuring credentials in Jenkins (Docker Hub credentials, SSH keys, MongoDB URI) was challenging. Incorrect credential IDs caused pipeline failures and prevented Docker images from being pushed to Docker Hub.

2. Docker Image Not Available in Docker Hub

Initially, Docker images were not available in Docker Hub, resulting in errors while pulling images on the AWS EC2 instance. This occurred because Jenkins failed to push images due to missing Docker Hub credentials.

3. Jenkins Pipeline Script Errors

The Jenkinsfile caused multiple syntax and compilation errors such as missing braces and incorrect shell script formatting. These errors stopped the pipeline execution and required careful debugging.

4. SSH Access and Permission Issues with AWS EC2

Connecting Jenkins to AWS EC2 using SSH keys was difficult. Incorrect key paths and permission settings resulted in authentication failures while deploying containers.

5. Backend Container Not Running on AWS

The frontend application failed to communicate with the backend because the backend container was not running on the EC2 instance. This caused `ERR_CONNECTION_REFUSED` errors during login requests.

6. Security Group Port Configuration

Initially, required ports such as 5000 (backend API) and 80 (frontend) were not open in the AWS security group, which prevented external access to the application.

7. Automatic Trigger Configuration in Jenkins

Setting up automatic build triggers using Poll SCM required correct scheduling. Incorrect configuration caused builds not to trigger automatically after pushing code to GitHub.

7. Conclusion

The Food Delivery Web Application project successfully demonstrates the practical implementation of DevOps principles in a real-world web application. The primary objective of this project was not only to build a functional food delivery system but also to design and implement a fully automated CI/CD pipeline that reflects modern industry practices.

By integrating tools such as Git, GitHub, Jenkins, Docker, and AWS EC2, the project achieves complete automation of the software delivery lifecycle. Jenkins plays a crucial role in automating builds and deployments, eliminating manual errors and ensuring consistent application updates. Docker ensures that the application runs in identical environments across development and production, solving the common “works on my machine” problem.

The deployment of the backend on AWS EC2 demonstrates cloud scalability and availability, while MongoDB Atlas provides a reliable and secure cloud-based database solution. Cloudinary enhances performance by managing image storage efficiently. The use of these tools collectively improves reliability, maintainability, and scalability of the system.

This project provides hands-on experience with DevOps tools widely used in the software industry. It bridges the gap between development and operations by emphasizing automation, collaboration, and continuous delivery. The system is scalable, efficient, and ready for future enhancements such as Kubernetes orchestration, monitoring tools, and advanced security mechanisms.

Overall, the project successfully fulfills its objectives and serves as a strong academic demonstration of DevOps-driven application development and deployment, making it highly relevant for modern software engineering practices.

8. REFERENCES

1. Docker Inc., *Docker Documentation*, 2024. [Online]. Available: <https://docs.docker.com>. [Accessed: 02-Nov-2025].
2. Jenkins Community, *Jenkins User Documentation*, 2024. [Online]. Available:
<https://www.jenkins.io/doc/>. [Accessed: 02-Nov-2025].
3. <https://www.jenkins.io/doc/>. [Accessed: 02-Nov-2025].
4. Amazon Web Services, *Amazon EC2 User Guide*, 2024. [Online]. Available:
<https://docs.aws.amazon.com/ec2/>. [Accessed: 10-Dec-2025].
5. <https://docs.aws.amazon.com/ec2/>. [Accessed: 10-Dec-2025].
6. MongoDB Inc., *MongoDB Atlas Cloud Database Documentation*, 2024. [Online]. Available: <https://www.mongodb.com/docs/atlas/>. [Accessed: 10-Dec2025].
7. OpenJS Foundation, *Node.js Official Documentation*, 2024. [Online]. Available: <https://nodejs.org/en/docs/>. [Accessed: 10-Dec-2025].
8. Express.js Team, *Express.js Framework Documentation*, 2024. [Online]. Available: <https://expressjs.com/>. [Accessed: 15-Dec-2025].
9. GitHub Inc., *GitHub Documentation – Version Control and CI/CD Integration*, 2024. [Online]. Available: <https://docs.github.com/>. [Accessed: 10Dec-2025].